

# Function – Triger - DCL

# Function

# Definition

- ❖ A database object that encapsulates logic and returns a value. It allows for modular, reusable code.
  
- ❖ **Function Types:**
  1. Scalar Function: Returns a single value (e.g., integer, string).
  2. Table-Valued Function (TVF): Returns a table of data.

# Advantages of Using Function

- ❖ **Code Reusability:** Functions allow you to reuse code across multiple queries.
- ❖ **Modular Design:** Break complex logic into smaller, manageable pieces.
- ❖ **Improved Query Readability:** Encapsulate complex calculations into simple function calls.
- ❖ **Performance:** Can be optimized for common repetitive calculations.

# Component Function

- ❖ Name: The Function's unique name.
- ❖ Parameters: **Optional** Input or output parameters.
- ❖ SQL Logic: The actual SQL statements inside the function.
- ❖ Return Type: **Mandatory** returns a value (using RETURN)

# Format Function

```
CREATE FUNCTION [function_name] (@param1  
datatype, @param2 datatype)  
RETURNS datatype  
AS  
BEGIN  
    -- SQL Statements  
    RETURN Result  
END;
```

# Scalar Function

**Aim:** Calculations like summing, averaging, string manipulations, date calculations, etc.

```
CREATE FUNCTION dbo.AddNumbers (@Num1 INT, @Num2 INT)
RETURNS INT
AS
BEGIN
    DECLARE @Result INT;
    SET @Result = @Num1 + @Num2;
    RETURN @Result;
END;
```

OR

```
CREATE FUNCTION dbo.AddNumbers (@Num1 INT, @Num2 INT)
RETURNS INT
AS
BEGIN
    RETURN (@Num1 + @Num2);
END;
```

# Scalar Function (2)

```
CREATE FUNCTION dbo.ValidateEmail (@email VARCHAR(255))
RETURNS BIT
AS
BEGIN
    DECLARE @isValid BIT;

    -- Check if the email is in a basic pattern: some_string@domain.com
    IF (@email LIKE '%_@__%.__%')
    BEGIN
        -- This pattern checks if there is an @ and a period in the email.
        -- Further validation could be added based on more complex patterns or rules.

        SET @isValid = 1;
    END
    ELSE
    BEGIN
        SET @isValid = 0;
    END

    RETURN @isValid;
END;
```

# Scalar Function (3)

## How to Use?

```
SELECT dbo.ValidateEmail('test@example.com') AS IsValidEmail; -- Returns 1
```

```
SELECT dbo.ValidateEmail('invalid-email') AS IsValidEmail; -- Returns 0
```

# Table-Valued Functions (TVF)

## Two Types of TVFs:

- 1. Inline Table-Valued Function (ITVF):** Returns the result of a single SELECT query.
- 2. Multi-Statement Table-Valued Function (MSTVF):** Allows multiple statements and returns a table variable.

```
CREATE FUNCTION dbo.GetEmployees (@DepartmentId INT)
RETURNS TABLE
AS
RETURN (
    SELECT EmployeeId, Name, DepartmentId
    FROM Employees
    WHERE DepartmentId = @DepartmentId
);
```

# MSTVF

**CREATE FUNCTION** dbo.GetEmployeeDetails (@DepartmentId INT)

**RETURNS** @EmployeeDetails TABLE

(

    EmployeeId INT,

    Name VARCHAR(100),

    Position VARCHAR(100)

)

**AS**

**BEGIN**

**INSERT INTO** @EmployeeDetails

**SELECT** EmployeeId, Name, Position

**FROM** Employees

**WHERE** DepartmentId = @DepartmentId;

**RETURN;**

**END;**

# Scalar vs TVF

| Aspect         | Scalar Function                                       | Table-Valued Function (TVF)                      |
|----------------|-------------------------------------------------------|--------------------------------------------------|
| Return Type    | Returns a single value (e.g., INT, VARCHAR, DATETIME) | Returns a table (with multiple rows and columns) |
| Use Case       | For simple calculations or transformations            | For queries that return multiple rows or tables  |
| Performance    | Can be slower when used in large queries (row-by-row) | Inline TVF typically performs better than MSTVF  |
| SQL Operations | Can be used in SELECT, WHERE, HAVING, etc.            | Used in FROM clause like a normal table          |

# Performance Considerations

## ❖ **Scalar UDFs:**

1. Can negatively impact performance if used in large queries (because they execute row-by-row).
2. Avoid using scalar UDFs in WHERE or JOIN clauses with large datasets.

## ❖ **TVFs:**

1. Inline TVFs typically perform better than multi-statement TVFs, as the result is computed in a single query.
2. Multi-statement TVFs can cause temporary table creation, which may affect performance.

# Performance Considerations

## ❖ **Scalar UDFs:**

1. Can negatively impact performance if used in large queries (because they execute row-by-row).
2. Avoid using scalar UDFs in WHERE or JOIN clauses with large datasets.

## ❖ **TVFs:**

1. Inline TVFs typically perform better than multi-statement TVFs, as the result is computed in a single query.
2. Multi-statement TVFs can cause temporary table creation, which may affect performance.

# Triger

# Definition

- ❖ A trigger is a database object in SQL Server that automatically performs a specified action when certain events occur on a specified table or view.
- ❖ Purpose: Triggers are often used for tasks like enforcing business rules, logging changes, and automating updates without requiring manual intervention.
- ❖ Events that trigger an action: INSERT, UPDATE, DELETE

# Types of Triggers

## A. DML Triggers (Data Manipulation Language)

1. **AFTER** Trigger: Executes after an INSERT, UPDATE, or DELETE operation.
2. **INSTEAD OF** Trigger: Executes instead of an INSERT, UPDATE, or DELETE operation, overriding the event.

## B. DDL Triggers (Data Definition Language)

Used to respond to database schema changes such as CREATE, ALTER, DROP.

## C. Logon/Logoff Triggers

Triggered when a user logs in or out of the SQL Server instance.

# Component Trigger

- ❖ Name: The Trigger's unique name.
- ❖ Object Handling: Table or Database.
- ❖ Type Event Handling: based event and type of Trigger
- ❖ SQL Logic: The actual SQL statements inside the function.

# Format Triger DML

```
CREATE TRIGGER [Triger_name]
ON [OBJECT HANDLING]
[Type] [EVENT HANDLING]
AS
BEGIN
    -- SQL Statements
END;
```

# DML Triger

```
CREATE TABLE StockOpname (  
    StockOpnameID INT PRIMARY KEY,  
    ProductID INT,  
    Quantity INT,  
    OpnameDate DATETIME  
);
```

```
CREATE TABLE Inventory (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100),  
    CurrentStock INT  
);
```

Now, let's create a trigger to automatically update the **Inventory** table when a new record is inserted into **StockOpname**.

**So, Need Triger After Insert**

# After Triger (DML Triger) (1)

```
CREATE TRIGGER trg_AfterStockOpnameInsert
ON StockOpname
AFTER INSERT
AS
BEGIN
    -- Declare variables to hold the inserted values
    DECLARE @ProductID INT;
    DECLARE @Quantity INT;

    -- Loop through the inserted rows (in case of multiple inserts)
    DECLARE inserted_cursor CURSOR FOR
        SELECT ProductID, Quantity
        FROM inserted;

    OPEN inserted_cursor;
    FETCH NEXT FROM inserted_cursor INTO @ProductID, @Quantity;

    -- Process each inserted row
    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- Update the inventory table based on the new stock opcode
        UPDATE Inventory
        SET CurrentStock = CurrentStock + @Quantity
        WHERE ProductID = @ProductID;

        -- Move to the next row in the inserted cursor
        FETCH NEXT FROM inserted_cursor INTO @ProductID, @Quantity;
    END

    -- Clean up the cursor
    CLOSE inserted_cursor;
    DEALLOCATE inserted_cursor;
END;
```

## After Triger (DML Triger) (2)

```
CREATE TRIGGER trg_AfterStockOpnameInsert
ON StockOpname
AFTER INSERT
AS
BEGIN
    -- Update inventory table using a set-based operation
    UPDATE Inventory
    SET CurrentStock = CurrentStock + i.Quantity
    FROM Inventory inv
    INNER JOIN inserted i
    ON inv.ProductID = i.ProductID;
END;
```

# Instead of Triger (DML Triger)

-- Single Row

```
CREATE TRIGGER trg_InsteadOfInsert_Product
ON Products
INSTEAD OF INSERT
AS
BEGIN
```

```
    -- Check if the Price is positive in the inserted rows
    IF EXISTS (SELECT 1 FROM inserted WHERE Price < 0)
        BEGIN
            -- If there's a negative price, raise an error and prevent insert
            RAISERROR('Price cannot be negative.', 16, 1);
        END
    ELSE
        BEGIN
            -- Otherwise, perform the insert operation as normal
            INSERT INTO Products (ProductID, ProductName, Price)
            SELECT ProductID, ProductName, Price
            FROM inserted;
        END
```

```
END;
```

# Instead of Triger (DML Triger) (2)

```
CREATE TRIGGER trg_AfterStockOpnameInsert
ON StockOpname
INSTEAD OF INSERT
AS
BEGIN
    -- Declare variables to hold the inserted values
    DECLARE @ProductID INT;
    DECLARE @Quantity INT;

    -- Loop through the inserted rows (in case of multiple inserts)
    DECLARE inserted_cursor CURSOR FOR
        SELECT ProductID, Quantity
        FROM inserted;

    OPEN inserted_cursor;
    FETCH NEXT FROM inserted_cursor INTO @ProductID, @Quantity;

    -- Process each inserted row
    WHILE @@FETCH_STATUS = 0
    BEGIN
        -- SCRIPT SQL to Check if Exist ProductId, then Commit
        -- SCRIPT SQL to Check if Not Exist ProductId, then RollBack

        -- Move to the next row in the inserted cursor
        FETCH NEXT FROM inserted_cursor INTO @ProductID, @Quantity;
    END

    -- Clean up the cursor
    CLOSE inserted_cursor;
    DEALLOCATE inserted_cursor;
END;
```

# DDL Trigger

```
CREATE TRIGGER DDL_AuditTrigger
ON DATABASE
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
AS
BEGIN
    DECLARE @EventType VARCHAR(100)
    DECLARE @ObjectName VARCHAR(255)
    DECLARE @ObjectType VARCHAR(100)
    DECLARE @SQLText NVARCHAR(MAX)

    -- Capture event details
    SET @EventType = EVENTDATA().value('(/EVENT_INSTANCE/EventType)[1]', 'VARCHAR(100)')
    SET @ObjectName = EVENTDATA().value('(/EVENT_INSTANCE/ObjectName)[1]', 'VARCHAR(255)')
    SET @ObjectType = EVENTDATA().value('(/EVENT_INSTANCE/ObjectType)[1]', 'VARCHAR(100)')
    SET @SQLText = EVENTDATA().value('(/EVENT_INSTANCE/TSQLCommand)[1]', 'NVARCHAR(MAX)')

    -- Log the event to the DDL_AuditLog table
    INSERT INTO DDL_AuditLog (EventTime, EventType, ObjectName, ObjectType, SQLText)
    VALUES (GETDATE(), @EventType, @ObjectName, @ObjectType, @SQLText)
END;
```

# Advantages of Using Trigger

1. Automation: Automates repetitive tasks, such as logging changes or updating related tables.
2. Data Integrity: Ensures consistency and enforces business rules.
3. Security and Auditing: Triggers can be used for tracking user actions, which helps with security audits.
4. No Application Code Required: Trigger actions are handled directly by the database, reducing reliance on application logic.

# Disadvantages of Using Trigger

1. **Performance Overhead**: Triggers can slow down operations, especially if they are complex or involve multiple triggers.
2. **Complexity**: Overuse or poorly designed triggers can complicate database maintenance and troubleshooting.
3. **Hidden Logic**: Triggers can hide business logic from the application layer, making it harder to debug.
4. **Recursion**: Triggers can potentially call themselves or other triggers, leading to infinite loops if not managed carefully.

# **Best Practices for Using Triggers**

1. Limit the Use of Triggers: Use them only when necessary to avoid performance issues.
2. Keep Trigger Logic Simple: Avoid complex operations in triggers to reduce overhead and maintain clarity.
3. Test Thoroughly: Ensure that triggers do not interfere with normal application functionality and performance.
4. Document Trigger Behavior: Clearly document the purpose and behavior of each trigger to ensure that it's easily understood by team members.
5. Avoid Recursive Triggers: Be cautious of triggers that may cause recursion, either by calling other triggers or themselves.

# Data Control Language (DCL)

# Definition

- ❖ DCL (Data Control Language) is a subset of SQL used to define and manage permissions and access control to data within a database.
- ❖ Purpose: DCL helps ensure security and authorization by granting or revoking privileges.
- ❖ Key Commands: GRANT, REVOKE, DENY

# Benefit DCL

1. Access Control: Provides fine-grained control over who can do what in the database.
2. Security: Protects sensitive data from unauthorized access.
3. Audit: Tracks access and changes made to the data for auditing purposes.

# Best Practice DCL

1. Least Privilege Principle: Only grant the minimum necessary privileges to users.
2. Regular Audits: Periodically review granted permissions to ensure they are still appropriate.
3. Role-Based Access Control (RBAC): Group users by roles and assign permissions to roles rather than individual users.
4. Avoid Overuse of DCL: Granting excessive privileges can increase the risk of unauthorized data access or accidental modification.

# Create or Drop User

## Case SQL SERVER

### A. Create User

1. Login As Admin
2. Create Script Login

```
CREATE LOGIN myLogin WITH PASSWORD = 'Password123!';
```

3. Choose Database

```
USE MyDatabase;
```

4. Create User Access

```
CREATE USER myUser FOR LOGIN myLogin;
```

### B. Drop User

1. Drop User

```
DROP USER myUser;
```

2. Drop Login

```
DROP LOGIN myLogin;
```

# Grant

The **GRANT** command is used to assign specific privileges to a user or a group of users.

1. Grant **SELECT** privilege to user 'myUser' on the 'employees' table  
`GRANT SELECT ON employees TO myUser;`
2. Grant **SELECT, INSERT, UPDATE, DELETE** privilege to user 'myUser' on the 'employees' table  
`GRANT SELECT, INSERT, UPDATE, DELETE ON employees TO myUser;`

# REVOKE

The **REVOKE** command removes previously granted privileges from a user or group.

1. Revoke **SELECT** privilege to user 'myUser' on the 'employees' table  
`REVOKE SELECT ON employees TO myUser;`
2. Revoke **UPDATE, DELETE** privilege to user 'myUser' on the 'employees' table  
`REVOKE UPDATE, DELETE ON employees TO myUser;`

# DENY

The **DENY** command explicitly denies a specific permission to a user, regardless of any other permissions that may have been granted to that user.

1. Deny **SELECT** privilege to user 'myUser' on the 'employees' table  
**DENY SELECT** ON employees TO myUser;
2. Deny **UPDATE, DELETE** privilege to user 'myUser' on the 'employees' table  
**DENY UPDATE, DELETE** ON employees TO myUser;

# DENY vs REVOKE

| Feature         | DENY                                                             | REVOKE                                                                                 |
|-----------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| Purpose         | Explicitly denies permission to the user, regardless of grants.  | Removes previously granted permission.                                                 |
| Precedence      | Takes precedence over GRANT permissions.                         | Removes GRANT but does not override DENY.                                              |
| Access Control  | Denies the permission even if the user has it granted elsewhere. | Simply removes the permission, it won't deny the user if the permission was inherited. |
| Impact on Roles | Denial cannot be overridden by roles.                            | Revoking a user's permission does not affect inherited permissions from roles.         |
| Use Case        | When you need to block access entirely to a specific permission. | When you need to revoke granted access, but not necessarily block it entirely.         |

# Exercise

1. Create Scalar Function on your Final Project
2. Create Table Value Function on your Final Project
3. Create Triger before Insert Update/Insert/Delete on your Final Project
4. Create Triger After Insert Update/Insert/Delete on your Final Project
5. Implement DCL with new User on your Final Project



Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,  
niscaya ilmu yang sedikit pun akan bermanfaat baginya